PIZZA SALES DATA ANALYSIS USING SQL

AUTHOR: AYUSH

TOOLS USED: MYSQL / SQL

PROJECT TYPE: DATA ANALYSIS PROJECT

HELLO!

Project Overview

This project analyzes pizza sales data using SQL to uncover
business insights. The analysis focuses on identifying revenue
trends, popular pizza types, customer ordering behavior, and
peak sales hours.

By using SQL queries on multiple tables, this project demonstrates
data analysis skills such as joins, aggregations, and grouping.

Dataset Description

The dataset used in this project contains pizza sales transaction data from a pizza restaurant. It includes information about orders, pizza types, pizza sizes, prices, and quantities sold. The data is organized into four tables: orders, order_details, pizzas, and pizza_types. These tables are connected using keys such as order_id and pizza_id to analyze sales performance and customer ordering patterns.

Business Questions

- Retrieve the total number of orders placed.
- Calculate the total revenue generated from pizza sales.
- Identify the highest-priced pizza.
- Identify the most common pizza size ordered.
- List the top 5 most ordered pizza types along with their quantities.
- Determine the distribution of orders by hour of the day.
- Calculate the average number of pizzas ordered per day
- Determine the top 3 most ordered pizza types based on revenue.

Calculate the total revenue generated from pizza sales.

```sql
SELECT
    ROUND(SUM(orders_details.quantity * pizzas.price), 2) AS total_revenue
FROM orders_details
JOIN pizzas
ON pizzas.pizza_id = orders_details.pizza_id;
```

Result Grid

total_revenue
817860.05

Identify the highest-priced pizza.

```
select pizza_types.name, pizzas.price
from pizza_types join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
order by pizzas.price desc limit 1;
```

Result Grid | Filter Rows:

name	price
The Greek Pizza	35.95

Identify the most common pizza size ordered.

```sql
SELECT pizzas.size,
       COUNT(orders_details.order_details_id) AS order_count
FROM pizzas
JOIN orders_details
ON pizzas.pizza_id = orders_details.pizza_id
GROUP BY pizzas.size order by order_count desc ;
```

Result Grid Filter Rows:

size	order_count
L	18526
M	15385
S	14137
XL	544
XXL	28

List the top 5 most ordered pizza types along with their quantities.

```sql
SELECT
    pizza_types.name,
    SUM(orders_details.quantity) AS total_quantity
FROM orders_details
JOIN pizzas
    ON pizzas.pizza_id = orders_details.pizza_id
JOIN pizza_types
    ON pizza_types.pizza_type_id = pizzas.pizza_type_id
GROUP BY pizza_types.name
ORDER BY total_quantity DESC
LIMIT 5;
```

Result Grid | Filter Rows:

name	total_quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

Join the necessary tables to find the total quantity of each pizza category ordered.

```sql
SELECT
    pizza_types.category,
    SUM(orders_details.quantity) AS total_quantity
FROM orders_details
JOIN pizzas
    ON pizzas.pizza_id = orders_details.pizza_id
JOIN pizza_types
    ON pizza_types.pizza_type_id = pizzas.pizza_type_id
GROUP BY pizza_types.category
ORDER BY total_quantity DESC;
```

Result Grid | Filter Rows:

category	total_quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

Determine the distribution of orders by hour of the day.

```sql
SELECT
    DATE_FORMAT(order_time, '%h %p') AS order_hour,
    COUNT(order_id) AS total_orders
FROM orders
GROUP BY order_hour
ORDER BY STR_TO_DATE(order_hour, '%h %p');
```

Result Grid Filter Rows:

order_hour	total_orders
09 AM	1
10 AM	8
11 AM	1231
12 PM	2520
01 PM	2455
02 PM	1472
03 PM	1468
04 PM	1920
05 PM	2336
06 PM	2399
07 PM	2009
08 PM	1642

Join relevant tables to find the category-wise distribution of pizzas.

```sql
SELECT
    pizza_types.category,
    COUNT(pizzas.pizza_id) AS total_pizzas
FROM pizzas
JOIN pizza_types
    ON pizzas.pizza_type_id = pizza_types.pizza_type_id
GROUP BY pizza_types.category
ORDER BY total_pizzas DESC;
```

Result Grid | Filter Rows:

category	total_pizzas
Veggie	27
Classic	26
Supreme	25
Chicken	18

Group the orders by date and calculate the average number of pizzas ordered per day.

```sql
SELECT
    ROUND(AVG(daily_total), 2) AS avg_pizzas_per_day
FROM (
    SELECT
        orders.order_date,
        SUM(orders_details.quantity) AS daily_total
    FROM orders
    JOIN orders_details
        ON orders.order_id = orders_details.order_id
    GROUP BY orders.order_date
) AS daily_orders;
```

Result Grid Filter Rows:

	avg_pizzas_per_day
▶	138.47

Determine the top 3 most ordered pizza types based on revenue.

```sql
SELECT
    pt.name AS pizza_type,
    ROUND(SUM(od.quantity * p.price), 2) AS total_revenue
FROM orders_details od
JOIN pizzas p
    ON od.pizza_id = p.pizza_id
JOIN pizza_types pt
    ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name
ORDER BY total_revenue DESC
LIMIT 3;
```

Result Grid | Filter Rows:

pizza_type	total_revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5

Calculate the percentage contribution of each pizza type to total revenue.

```sql
SELECT
    pt.category,
    ROUND(
        SUM(od.quantity * p.price) * 100
        / SUM(SUM(od.quantity * p.price)) OVER (),
    2) AS revenue_percentage
FROM orders_details od
JOIN pizzas p
    ON od.pizza_id = p.pizza_id
JOIN pizza_types pt
    ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category
ORDER BY revenue_percentage DESC;
```

Result Grid | Filter Rows:

category	revenue_percentage
Classic	26.91
Supreme	25.46
Chicken	23.96
Veggie	23.68

Analyze the cumulative revenue generated over time.

```sql
SELECT
    order_date,
    daily_revenue,
    ROUND(
        SUM(daily_revenue) OVER (ORDER BY order_date),
        2
    ) AS cumulative_revenue
FROM (
    SELECT
        orders.order_date,
        SUM(orders_details.quantity * pizzas.price) AS daily_revenue
    FROM orders
    JOIN orders_details
        ON orders.order_id = orders_details.order_id
    JOIN pizzas
        ON pizzas.pizza_id = orders_details.pizza_id
    GROUP BY orders.order_date
) AS daily_sales;
```

Result Grid | Filter Rows:

order_date	daily_revenue	cumulative_revenue
2015-01-01	2713.85	2713.85
2015-01-02	2731.9	5445.75
2015-01-03	2662.4	8108.15
2015-01-04	1755.45	9863.6
2015-01-05	2065.95	11929.55
2015-01-06	2428.95	14358.5
2015-01-07	2202.2	16560.7
2015-01-08	2838.35	19399.05
2015-01-09	2127.35	21526.4
2015-01-10	2463.95	23990.35
2015-01-11	1872.3	25862.65
2015-01-12	1919.05	27781.7

THANK YOU